

JavaScript → TypeScript

Maîtriser JavaScript moderne pour mieux comprendre TypeScript

JavaScript	TypeScript	ES6+	Closures
------------	------------	------	----------

Prototypes	Async	Types
------------	-------	-------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Maîtriser les fondamentaux de JavaScript moderne (ES6+) souvent mal compris
- ✓ Comprendre les mécanismes sous-jacents de JS : coercion, closures, prototype chain
- ✓ Identifier les classes d'erreurs que TypeScript prévient et pourquoi
- ✓ Réaliser une migration progressive d'un fichier JavaScript vers TypeScript
- ✓ Lire et écrire tous les constructs TypeScript courants sans avoir besoin de documentation
- ✓ Appliquer des patterns TypeScript idiomatiques dans un projet React Native + Node.js

Table des matières

Chapitre 1 — JavaScript moderne — les fondamentaux revisités

- 1.1 Variables : var, let, const
- 1.2 Fonctions : déclarations, expressions, fléchées
- 1.3 Closures et portée
- 1.4 Destructuration et spread

Chapitre 2 — Le système de types de JavaScript

- 2.1 Types primitifs et typeof
- 2.2 Coercition implicite — le piège
- 2.3 Equality : == vs ===
- 2.4 null, undefined et le optional chaining

Chapitre 3 — Asynchronisme en JavaScript

- 3.1 L'event loop revisitée
- 3.2 Callbacks et callback hell
- 3.3 Promises — la chaîne .then()
- 3.4 Async/Await — syntaxe et erreurs

Chapitre 4 — TypeScript — ce que JS ne peut pas faire

- 4.1 Le problème que TS résout
- 4.2 Inférence de types
- 4.3 Narrowing et type guards
- 4.4 Types génériques décodés

Chapitre 5 — Migration et patterns professionnels

- 5.1 Stratégie de migration JS→TS
- 5.2 any vs unknown vs never
- 5.3 Satisfies operator

5.4 Patterns idiomatiques

JavaScript moderne — les fondamentaux revisités

1.1 Variables : var, let, const

La déclaration de variables en JavaScript a évolué de `var` (ES5) vers `let` et `const` (ES6, 2015). Cette évolution n'est pas cosmétique — elle corrige des comportements contre-intuitifs de `var` qui sont une source fréquente de bugs.

`var` a une portée de fonction (function scope) : une variable déclarée avec `var` dans un bloc `if` ou `for` est accessible dans toute la fonction englobante. `let` et `const` ont une portée de bloc (block scope) : elles n'existent que dans le bloc `{}` où elles sont déclarées. `var` est aussi sujet au 'hoisting' : la déclaration est remontée au sommet de la fonction, mais pas l'initialisation — la variable existe (avec la valeur `undefined`) avant même sa ligne de déclaration.

var vs let vs const — les différences clés

```
// var – portée de fonction (comportement surprenant)

function example() {
  for (var i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100);
  }
}

example(); // Affiche : 3, 3, 3 – pas 0, 1, 2 !

// Pourquoi ? 'i' est partagé entre tous les callbacks (même variable)

// let – portée de bloc (comportement attendu)

function example2() {
  for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 100);
  }
}

example2(); // Affiche : 0, 1, 2 – chaque itération a son propre 'i'

// const – référence immuable (pas la valeur pour les objets)

const user = { name: 'Mohamed' };

user.name = 'Fomba'; // ✓ OK – on modifie le contenu de l'objet

user = { name: 'Autre' }; // ■ Erreur – on ne peut pas réassigner la variable
```

1.2 Fonctions fléchées et this

Les fonctions fléchées (arrow functions, ES6) sont plus qu'une syntaxe raccourcie. Elles ne créent pas leur propre contexte `this` — elles capturent le `this` du contexte englobant. C'est une différence fondamentale avec les fonctions classiques, et la source de nombreux bugs en JavaScript.

```
// Fonction classique – this dépend de l'appelant

const timer = {
  seconds: 0,
  start: function() {
    setInterval(function() {
      this.seconds++; // this = window (ou undefined en strict mode) – bug !
    }, 1000);
  }
};

// Fonction fléchée – this lexical (hérite du contexte parent)

const timer2 = {
  seconds: 0,
  start: function() {
    setInterval(() => {
      this.seconds++; // this = timer2 – correct !
    }, 1000);
  }
};

// En React/React Native – les callbacks doivent être des fléchées
// Pour que 'this' (ou les variables fermées) soient correctes
```

1.3 Closures et portée

Une closure est une fonction qui 'se souvient' des variables de sa portée de définition, même après que la portée extérieure ait été quittée. C'est l'un des concepts les plus importants (et les plus mal compris) de JavaScript. Les hooks React (`useState`, `useEffect`, `useCallback`) sont basés sur les closures.

```
// Closure – la fonction 'se souvient' de count

function createCounter() {
  let count = 0; // Variable privée – accessible seulement via les closures
  return {
    increment: () => ++count,
```

```
decrement: () => --count,
value: () => count,
};
}

const counter = createCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.decrement(); // 1
counter.value(); // 1
// 'count' n'est pas accessible directement depuis l'extérieur

// En React – useCallback crée une closure sur les variables du composant
const handleSubmit = useCallback(async () => {
  // 'email' et 'password' sont capturés depuis le state
  await signIn(email, password);
}, [email, password]); // Recréer si email ou password change
```

Le système de types de JavaScript

2.1 Types primitifs et typeof

JavaScript possède huit types primitifs : string, number, bigint, boolean, undefined, symbol, null, et object (qui inclut les arrays, les fonctions, et null — c'est un bug historique). L'opérateur typeof retourne le type d'une valeur sous forme de chaîne de caractères. Ses résultats sont parfois surprenants.

```
typeof 'hello' // 'string'
typeof 42 // 'number'
typeof true // 'boolean'
typeof undefined // 'undefined'
typeof {} // 'object'
typeof [] // 'object' – pas 'array' !
typeof null // 'object' – bug historique de JS
typeof function(){} // 'function'

// Vérifications correctes
Array.isArray([]) // true
value === null // seule façon fiable de tester null
value instanceof Date // tester une instance de classe
```

2.2 Coercition implicite — le piège

La coercition implicite est le mécanisme par lequel JavaScript convertit automatiquement des valeurs d'un type vers un autre dans certaines opérations. C'est l'une des sources de bugs les plus fréquentes en JavaScript, et c'est l'une des raisons principales pour lesquelles TypeScript a été créé.

```
// Coercition avec l'opérateur +
'5' + 3 // '53' – number converti en string
5 + '3' // '53' – idem
5 - '3' // 2 – string converti en number (- ne fait pas de concat)
true + true // 2 – boolean converti en number
'5' - '3' // 2 – les deux convertis en number

// Coercition avec == (comparaison lâche)
0 == false // true – false converti en 0
```



```
'' == false // true – les deux falsies
null == undefined // true – cas spécial
[] == false // true – tableau vide est falsy

// Solution : toujours utiliser === (comparaison stricte)
0 === false // false – types différents, pas de coercition

// TypeScript prévient ces erreurs à la compilation
// '5' + 3 → erreur si les types sont annotés correctement
```

Asynchronisme en JavaScript

3.1 Promises — la chaîne .then()

Une Promise est un objet représentant la complétion ou l'échec futur d'une opération asynchrone. Elle peut être dans trois états : pending (en attente), fulfilled (résolue avec succès), ou rejected (rejetée avec une erreur). Les méthodes .then() et .catch() permettent de chaîner des opérations asynchrones de façon lisible.

```
// Créer une Promise

function fetchUser(id: string): Promise<User> {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id === 'invalid') {
        reject(new Error('User not found'));
      } else {
        resolve({ id, name: 'Mohamed' });
      }
    }, 100);
  });
}

// Chaîner des Promises

fetchUser('123')
  .then(user => fetchOrders(user.id)) // retourne une nouvelle Promise
  .then(orders => console.log(orders))
  .catch(err => console.error(err)) // capture TOUTES les erreurs en amont
  .finally(() => setLoading(false)); // exécuté dans tous les cas

// Promise.all – exécution parallèle

const [users, products] = await Promise.all([
  fetchUsers(tenantId),
  fetchProducts(tenantId),
]);

// Échoue si L'UNE des promises échoue

// Promise.allSettled – toutes doivent se terminer
```

```
const results = await Promise.allSettled([fetchA(), fetchB(), fetchC()]);  
  
// Ne rejette jamais – chaque résultat indique {status: 'fulfilled'/'rejected'}
```

3.2 Pièges async/await courants

Async/await simplifie l'écriture du code asynchrone mais introduit ses propres pièges. Le plus fréquent est l'attente séquentielle inutile : écrire await pour deux requêtes indépendantes les une après l'autre est deux fois plus lent que de les lancer en parallèle avec Promise.all.

```
// ■ Anti-pattern : attente séquentielle inutile  
  
async function loadDashboard(tenantId: string) {  
  const orders = await fetchOrders(tenantId); // 300ms  
  const products = await fetchProducts(tenantId); // 200ms  
  const stats = await fetchStats(tenantId); // 150ms  
  
// Total : ~650ms  
}  
  
// ■ Correctif : exécution parallèle  
  
async function loadDashboard(tenantId: string) {  
  const [orders, products, stats] = await Promise.all([  
    fetchOrders(tenantId),  
    fetchProducts(tenantId),  
    fetchStats(tenantId),  
  ]);  
  
// Total : ~300ms (le plus lent des trois)  
}  
  
// ■ Oublier le try/catch – crash silencieux  
  
async function loadUser(id: string) {  
  const user = await fetchUser(id); // Peut rejeter – exception non catchée !  
  return user;  
}  
  
// ■ Toujours gérer les erreurs  
  
async function loadUser(id: string) {  
  try {  
    return await fetchUser(id);  
  } catch (err) {  
    throw new NotFoundError('User not found');  
  }  
}
```


TypeScript — ce que JS ne peut pas faire

4.1 Inférence de types

L'un des malentendus sur TypeScript est qu'il faut annoter chaque variable. En réalité, TypeScript infère la grande majorité des types automatiquement. Quand vous écrivez `const x = 42`, TypeScript sait que `x` est un `number`. Annoter explicitement `const x: number = 42` est redondant. L'annotation explicite est nécessaire uniquement quand l'inférence est insuffisante : les paramètres de fonctions, les retours ambigus, les variables initialisées plus tard.

```
// TypeScript INFÈRE ces types – pas besoin d'annotation
const name = 'Mohamed'; // string
const age = 28; // number
const tags = ['vip', 'pro']; // string[]
const user = { id: '1', email: 'a@b.com' }; // { id: string, email: string }

// Annotation nécessaire : paramètres de fonction
function greet(name: string): string {
  return `Bonjour ${name}`;
}

// Annotation nécessaire : type de retour complexe
function parseConfig(raw: unknown): Config {
  // ...
}

// Annotation nécessaire : variable initialisée plus tard
let currentUser: User | null = null;

// Sans annotation, TS l'inférerait comme 'null' – trop restrictif
```

4.2 Narrowing et type guards

Le type narrowing est la capacité de TypeScript à réduire un type large vers un type plus précis dans un bloc conditionnel. C'est ce qui rend les union types si puissants : TypeScript sait exactement quel type est disponible dans chaque branche d'un `if/switch`.

```
// Union type : string | number
function formatValue(val: string | number): string {
```

```
if (typeof val === 'string') {  
  // TypeScript sait que val est un string ici  
  return val.toUpperCase();  
}  
// TypeScript sait que val est un number ici  
return val.toFixed(2);  
}  
  
// Discriminated union – narrows sur une propriété discriminante  
type ApiResult<T> =  
  | { ok: true; data: T }  
  | { ok: false; error: string };  
  
function process<T>(result: ApiResult<T>): T {  
  if (result.ok) {  
    return result.data; // TypeScript sait que .data existe  
  }  
  throw new Error(result.error); // et que .error existe  
}  
  
// Type guard personnalisé  
function isOrder(val: unknown): val is Order {  
  return (  
    typeof val === 'object' && val !== null &&  
    'id' in val && 'status' in val && 'total_amount' in val  
  );  
}
```

Migration et patterns professionnels

5.1 Stratégie de migration JS→TS

Migrer un projet JavaScript existant vers TypeScript n'implique pas de tout réécrire d'un coup. La stratégie incrémentale est de renommer les fichiers un par un (.js → .ts), corriger les erreurs TypeScript en commençant par les moins critiques, et augmenter progressivement le niveau de strictness dans tsconfig.json. TypeScript accepte any pour les types inconnus — utiliser any temporairement pendant la migration, puis remplacer progressivement par des types précis.

```
# Étape 1 : Ajouter TypeScript sans casser quoi que ce soit
npm install -D typescript @types/node
npx tsc --init # Crée tsconfig.json avec strict: false

# Étape 2 : Renommer les fichiers les plus simples en .ts
mv src/lib/utils.js src/lib/utils.ts
npx tsc --noEmit # Voir les erreurs

# Étape 3 : Corriger les erreurs en annotant les fonctions
# Utiliser 'any' temporairement pour les cas difficiles

# Étape 4 : Augmenter la strictness progressivement
# tsconfig.json : ajouter 'noImplicitAny: true'
# Corriger toutes les erreurs 'Parameter x implicitly has an any type'

# Étape 5 : Activer 'strict: true' – objectif final
```

5.2 any vs unknown vs never

Ces trois types spéciaux sont souvent confondus. any désactive toute vérification — c'est JavaScript pur. unknown représente une valeur dont le type est inconnu à la compilation — vous devez vérifier le type avant de l'utiliser. never représente une valeur qui ne peut jamais exister — le type de retour d'une fonction qui throw toujours, ou les branches mortes d'un switch exhaustif.

```
// any – désactive le typage (éviter sauf migration)
let x: any = 'hello';
x.nonexistent.method(); // Aucune erreur TypeScript – crash à l'exécution

// unknown – type sûr pour les valeurs inconnues
```

```

let input: unknown = getUserInput();
input.toUpperCase(); // ■ Erreur – doit vérifier d'abord
if (typeof input === 'string') {
input.toUpperCase(); // ✓ OK – TS sait que c'est une string
}

// never – valeur impossible
function assertNever(val: never): never {
throw new Error(`Unhandled case: ${val}`);
}

// Pattern exhaustif avec never
function handleStatus(status: OrderStatus): string {
switch (status) {
case 'new': return 'Nouvelle';
case 'confirmed': return 'Confirmée';
case 'delivered': return 'Livrée';
case 'cancelled': return 'Annulée';
default:
return assertNever(status); // Erreur si un statut n'est pas géré
}
}

```

■ À retenir

- ✓ let et const remplacent var — portée de bloc, pas de hoisting surprenant
- ✓ Les fonctions fléchées capturent le this lexical — préférer en React/callbacks
- ✓ La coercition implicite est la principale raison d'être de TypeScript
- ✓ TypeScript infère la majorité des types — annotez seulement quand nécessaire
- ✓ Les discriminated unions + narrowing = pattern le plus puissant de TypeScript
- ✓ Migration incrémentale : renommer .js→.ts un par un, any temporairement autorisé

—■ Exercices

Exercice 1 — Débogage JavaScript

Identifier et corriger des bugs liés aux types JavaScript.

- Créer un fichier debug.js avec 5 bugs de coercition implicite
- Prédire la valeur de chacun AVANT d'exécuter
- Vérifier avec Node.js
- Réécrire en TypeScript strict pour que ces bugs soient détectés à la compilation

Exercice 2 — Migration fichier par fichier

Migrer un module JavaScript vers TypeScript complet.

- Renommer `utils.js` en `utils.ts`
- Corriger toutes les erreurs TypeScript sans utiliser `any`
- Ajouter des types de retour explicites à toutes les fonctions exportées
- Vérifier avec `tsc --noEmit : 0 erreur`

Exercice 3 — Async avancé

Réécrire des fonctions séquentielles en parallèle.

- Identifier les appels `await` qui peuvent être parallélisés
- Utiliser `Promise.all` pour les exécuter simultanément
- Mesurer l'amélioration de performance avec `Date.now()`
- Ajouter des types précis pour toutes les Promises

Bibliographie & Ressources

JavaScript

- javascript.info — guide JavaScript moderne et complet (français disponible)
- eloquentjavascript.net — livre Eloquent JavaScript (gratuit)
- MDN Web Docs — developer.mozilla.org — référence officielle

TypeScript

- typescriptrlang.org/docs/handbook — le Handbook TypeScript
- totaltypescript.com — Matt Pocock, exercices TypeScript avancés
- executeprogram.com/courses/typescript — cours interactifs